

Taller de búsquedas con información

Informe final - Conmutación de mensajes con Dijkstra

Dani-Sunshiny · Juliana Gutiérrez Rodríguez (JuliGR05)
Johan Santiago Camargo Olmos (bardock7238) · Mariana Castiblanco Bedoya
(mariana0110-hub)

Punto 1 y integración: Dani · Problema, pesos, mensajes y análisis: Juliana (#17-#20)
Pseudocódigo e informe final: Johan (#21, #28) · Red y escenarios: Mariana (#24-#26)
Diseño común: issue #15 · Código: paquete conmutacion, rama develop

Documentos: docs/punto1-dijkstra-rumania.md · docs/punto2-algoritmo-simulacion.md
Este informe: docs/informe-final.md (y este PDF)

1. Lo del punto 1, en corto

El punto 1 no tenía código, era lápiz y papel (bueno, tabla y cola de prioridad). Tomamos el grafo clásico de Rumania, con sus 25 carreteras y sus kilómetros reales, y corrimos Dijkstra a mano desde Arad hasta Bucharest. El detalle iteración por iteración está en docs/punto1-dijkstra-rumania.md; aquí va lo que importa para el resto del informe.

Se arranca con Arad en 0 y todo lo demás en infinito, y en cada iteración se saca el nodo con menor distancia y se relajan sus vecinos. Nos tomó 13 iteraciones llegar a Bucharest. Lo interesante es que Bucharest apareció primero con 468 km (por Fagaras), después bajó a 446 (por Craiova) y al final quedó en 418. Si nos hubiéramos quedado con el primer camino encontrado, la respuesta habría quedado mal.

La ruta que quedó fue: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest = 140 + 80 + 97 + 101 = 418 km, contra la alternativa por Fagaras (140 + 99 + 229 = 468 km). 50 km de diferencia.

Lo otro que nos quedó claro, y que después nos sirvió para el punto 2, es que Dijkstra no trabaja con distancia en línea recta sino con el costo real de cada arista. Sibiu, por ejemplo, está a 187 km de Bucharest en línea recta, pero el camino real más corto desde ahí cuesta 278 km. Esa diferencia entre "lo que parece cerca" y "lo que cuesta de verdad" es justo lo que la función de peso trata de capturar en la red de conmutación.

2. El problema que nos tocó simular

2.1 Qué entra y qué sale

La red es un grafo no dirigido. Cada enlace guarda latencia, congestión (0 a 1), probabilidad de caída (0 a 1), throughput y si está activo o no. Los mensajes se definen con origen, destino y número de paquetes, y además hay eventos que cambian la red a mitad de la simulación: algo como "después del paquete 2, el enlace B-R3 se congestiona" o "se cae el enlace A-R1".

Lo que el programa devuelve, por cada paquete, es por dónde se fue y cuándo llegó, y al final el orden global de llegada. Eso lo imprime SalidaConsola.

Las reglas que acordamos (issue #15) son sencillas: los paquetes salen uno por uno, uno cada 1.0 de tiempo, mensaje por mensaje. Para cada paquete se calcula la ruta con los pesos de ese momento, y el tiempo de llegada es el tiempo de envío más lo que cueste la ruta. El evento se aplica justo después del paquete que dice su número. Si no hay camino, el paquete no llega (ruta vacía y tiempo infinito) y queda de último. Cada corrida arranca con la red limpia, para que una simulación no contamine la siguiente.

2.2 Cómo se pesa cada enlace

La fórmula está en CalculadoraPeso.calcular y hace lo que uno esperaría: si el enlace está caído, pesa infinito y no se usa. Si no, se suman el retardo (latencia ajustada por congestión), una penalización por probabilidad de caída y otra por throughput bajo, y el total se multiplica por 2 si el enlace es de borde.

```
peso = factorZona * (latencia * (1 + congestión) + 100 * probCaída + 100 / throughput)
```

factorZona = 2.0 si es de borde, 1.0 si es de núcleo

Con los datos de nuestra red eso da, por ejemplo:

Enlace	Zona	Peso
R1-R2 (núcleo)	Núcleo	6.20
A-R1 (borde)	Borde	32.00
B-R3 con congestión 0.5	Borde	42.00

Se nota la intención del diseño: aunque un enlace de borde tenga latencia parecida a uno de núcleo, termina pesando unas cinco veces más. Y cuando se congestiona sube otros 10 puntos, lo suficiente para que el algoritmo prefiera rodear.

2.3 El algoritmo (lo esencial)

El pseudocódigo completo quedó en docs/punto2-algoritmo-simulacion.md. Aquí lo esencial: el simulador, por cada paquete, le pone su tiempo de envío, corre Dijkstra con los pesos actuales, le guarda la ruta y le calcula la llegada, y después aplica los eventos que toquen. Al final ordena todo por llegada.

Lo clave es que Dijkstra se corre una vez por paquete, no una vez por mensaje. Cada ejecución arranca sus vectores de distancia y predecesor desde cero, con una cola de prioridad, y va relajando vecinos hasta cubrir la red. Como los pesos son no negativos y el caído vale infinito (se salta), el algoritmo termina siempre. La complejidad por corrida es $O((V+E) \log V)$; en nuestra red (7 nodos, 11 enlaces) eso no es nada, pero como se repite por paquete, el total anda por $O(P * (V+E) \log V)$ más el ordenamiento final.

2.4 La red que armamos

La red la construye Main.construirRed: cuatro routers de núcleo (R1 a R4) bien conectados entre sí, y tres usuarios de borde (A, B, C), cada uno colgado de dos routers distintos para que siempre haya por dónde rodear. Los enlaces de núcleo tienen mejores números en todo y los de borde peores, como pedía el enunciado. En clases eso son ZonaRed, Nodo, Enlace y Red, con Main armando todo. No hubo sorpresas porque el #15 dejaba las firmas claras y cada quien las respetó.

Enlace	Lat.	Caída	Thr.	Peso
R1-R2	5	0.01	500	6.20
R1-R3	6	0.01	500	7.20
R2-R3	4	0.01	500	5.20
R2-R4	6	0.01	500	7.20
R3-R4	5	0.01	500	6.20
A-R1	10	0.05	100	32.00
A-R2	12	0.05	80	36.50
B-R3	10	0.05	100	32.00
B-R4	12	0.05	80	36.50

C-R2	10	0.05	100	32.00
C-R4	12	0.05	80	36.50

3. Qué pasó al correr los escenarios

Compilamos con `javac -encoding UTF-8 -d out conmutacion/*.java` y corrimos `conmutacion.Main`. Esto es lo que salió, copiado de la consola.

Escenario A: todo estable (A -> B, 5 paquetes, sin eventos)

Todos se fueron por el mismo lado, como era de esperar. La ruta cuesta 71.2 ($32.0 + 7.2 + 32.0$) y como sale un paquete por unidad de tiempo, las llegadas van 71.2, 72.2, etc. Sin cambios de peso no hay nada que recalcular.

Orden	Paquete	Llegada	Ruta
1	M1-P1	71.2	A -> R1 -> R3 -> B
2	M1-P2	72.2	A -> R1 -> R3 -> B
3	M1-P3	73.2	A -> R1 -> R3 -> B
4	M1-P4	74.2	A -> R1 -> R3 -> B
5	M1-P5	75.2	A -> R1 -> R3 -> B

Escenario B: congestión en B-R3 después del paquete 2 (A->B con 4 + C->B con 3)

Los dos primeros paquetes alcanzan a pasar por la ruta buena, pero del tercero en adelante el simulador ya ve el peso nuevo ($32 \rightarrow 42$) y manda todo por A -> R2 -> R4 -> B (80.2 de costo). Los paquetes de C, que se enviaron después, llegan antes que la cola de A porque su ruta (C -> R4 -> B, 73.0) es más barata. El orden de llegada ya no es el de envío, y eso está bien.

Orden	Paquete	Llegada	Ruta
1	M1-P1	71.2	A -> R1 -> R3 -> B
2	M1-P2	72.2	A -> R1 -> R3 -> B
3	M2-P1	77.0	C -> R4 -> B
4	M2-P2	78.0	C -> R4 -> B
5	M2-P3	79.0	C -> R4 -> B
6	M1-P3	82.2	A -> R2 -> R4 -> B
7	M1-P4	83.2	A -> R2 -> R4 -> B

Escenario C: se cae A-R1 después del paquete 2 (A->B, 5 paquetes)

Los dos primeros pasan normal; al caerse A-R1, el resto se va por A -> R2 -> R3 -> B ($36.5 + 5.2 + 32.0 = 73.7$). Funciona porque el enlace caído pesa infinito y Dijkstra lo ignora.

Orden	Paquete	Llegada	Ruta
1	M1-P1	71.2	A -> R1 -> R3 -> B
2	M1-P2	72.2	A -> R1 -> R3 -> B

3	M1-P3	75.7	A -> R2 -> R3 -> B
4	M1-P4	76.7	A -> R2 -> R3 -> B
5	M1-P5	77.7	A -> R2 -> R3 -> B

En resumen: el A confirma que la base está bien calibrada, el B muestra que una congestión parcial alcanza para desviar tráfico, y el C que una caída total también se maneja sin romper nada. En los tres, paquetes del mismo mensaje terminan con rutas distintas cuando hay eventos de por medio, que era lo pedido.

4. Revisión antes de cerrar

- Juntamos las partes en orden (punto 1; punto 2: problema, pesos, algoritmo, red, resultados) en este informe.
- Crucé el informe con el código: la fórmula es la de CalculadoraPeso.java, los tiempos y eventos son los de Simulador.java, y las salidas son las que bota Main de verdad.
- Los nombres de clases y métodos coinciden con el #15 (paquete conmutacion, sin tildes).
- Queda pendiente que cada uno le pegue una leída a la sección de otro antes de entregar (yo revisé red y escenarios; falta una pasada a Dijkstra/Simulador).
- Los dos puntos del taller quedan cubiertos: punto 1 (§1) y punto 2 (§2-§3).

Referencias

- docs/punto1-dijkstra-rumania.md - traza manual Arad -> Bucharest (#16).
- docs/punto2-algoritmo-simulacion.md - pseudocódigo y complejidad (#21).
- Issues: #15 (diseño), #16-#20 (P1-P2), #21-#23 y #28 (P3), #24-#26 (P4).
- Código en conmutacion/: ZonaRed, Nodo, Enlace, Red, CalculadoraPeso, Paquete, Mensaje, Evento, Dijkstra, Simulador, SalidaConsola, Main.
- Cormen et al., Introduction to Algorithms (Dijkstra con cola de prioridad).